

Why Agentic AI Fails: 循环、幻觉式规划与危险工具调用

IBM Technology & Codex

2026 年 6 月 10 日



视频频道: IBM Technology

发布日期: 2026 年 5 月 14 日

视频时长: 12 分 45 秒

视频链接: <https://www.youtube.com/watch?v=D37Ijn2o5U0>

主要证据: YouTube 手动英文字幕、原始封面、1080p 视频帧。

制作说明: 本文以中文讲解为主, 保留关键英文原句作为概念证据。

目录

1	从“模型幻觉”到系统设计	3
1.1	为什么失败常被归因给 hallucination	3
1.2	Agentic AI 的宽口径定义	4
1.3	失败面为什么从 chatbot 扩大到 agentic system	5
1.4	本章小结	5
2	失败模式一：Infinite Loop（无限循环）	5
2.1	文档搜索例子：search、evaluate、plan、retry	6
2.2	根因一：缺少 termination condition	7
2.3	根因二：缺少 action tracking	7
2.4	根因三：缺少 progress tracking	7
2.5	缓解：上限、差异比较与进度指标	8
2.6	可复用失败模式卡片	9
2.7	本章小结	9
3	失败模式二：Hallucinated Planning（幻觉式规划）	9
3.1	米兰机票例子：漂亮计划与真实能力	9
3.2	根因一：tool capabilities 没有清晰定义	10
3.3	根因二：planning 与 execution 缺少 validation gate	10
3.4	根因三：先假设能力，后考虑 constraints	11
3.5	缓解：清晰 schema、verifier agent、人类复核	11
3.6	可复用失败模式卡片	12
3.7	本章小结	12
4	失败模式三：Unsafe Tool Use（不安全工具使用）	13
4.1	数据库例子：删除 outdated records 时误删 active records	13
4.2	邮件例子：未 review 的 autonomous emails	13
4.3	根因一：overprivileged tools	14
4.4	根因二：approval workflow 缺失	14
4.5	根因三：read、write、delete 权限层级混在一起	14
4.6	风险门：把安全性做成可执行规则	14
4.7	缓解：最小代理性、审批流、权限分层	15
4.8	可复用失败模式卡片	16
4.9	本章小结	16
5	总结与工程检查清单	16

5.1	三类失败的共同结构	16
5.2	too much autonomy or too little constraint	17
5.3	monitoring 与 tracking 的底层作用	18
5.4	从 prompt 调参走向系统可靠性	18
5.5	面向真实项目的检查清单	18
5.6	本章小结	19

1 从“模型幻觉”到系统设计

这段视频的开场先抓住一个很常见的工程反应：当 agentic AI system 失败时，很多人第一句话会是 “*the model probably hallucinated*”，也就是“模型大概幻觉了” [00:00:00–00:00:06]。这个判断有现实基础。早期大语言模型经常表现不稳定，讲者用 “*probabilistic models*” 和 “*not deterministic*” 概括这种性质 [00:00:15–00:00:17]：同一个输入在不同采样、不同上下文、不同温度设置下可能得到不同输出。

本视频的核心口径随后转向系统层：近几年模型架构和工程化能力提高以后，许多 agentic AI 失败更应该被追踪到 “*flaws in the system design*”，也就是系统设计缺陷 [00:00:34–00:00:38]。这里的“系统”包括模型、工具、状态、约束、计划、执行、评估、重试、审批和监控。把失败全压到“模型幻觉”这个标签上，会遮住真正可修的工程问题。

本章核心判断

Agentic AI 可靠性首先是系统控制问题。LLM 是其中一个组件；停止条件、工具权限、计划验证、进度追踪和人工审批同样决定系统能否稳定完成任务。

1.1 为什么失败常被归因给 hallucination

Hallucination 在中文里通常译为“幻觉”，指模型生成了看似合理、实际缺乏可靠依据的内容。它像一个讲得很流畅的学生，语气自信，细节完整，答案却没有证据支撑。这个概念解释了很多纯聊天场景里的错误，所以当 agentic AI 出错时，人们自然沿用这套诊断。

需要分清两个相关概念。一个是“概率性”：模型从概率分布中产生输出，可以写成一个简化抽象：

$$y \sim p_{\theta}(y \mid x, c)$$

- y ：模型生成的输出，例如一句回答、一个计划步骤或一个工具调用参数。
- p_{θ} ：由参数 θ 表示的模型概率分布。
- x ：当前输入，例如用户任务。
- c ：上下文，包括历史对话、系统提示、工具返回、约束说明等。
- $\sim$ ：表示“从某个分布中采样产生”。

另一个是“确定性”：相同输入和相同系统状态总是得到相同输出。讲者说 “*not deterministic*” [00:00:16–00:00:17]，强调的是模型输出天然带有概率性。可是 agentic AI 的失败面更宽。一个模型回答本身可以合理，系统仍然可能因为重试规则、工具权限或审批流程缺失而失败。

概率模型与工程系统的差别

概率模型像一个会给出建议的顾问；工程系统像一个带着钥匙、清单和预算执行任务的初级操作员。顾问说错话会造成认知错误；操作员拿错钥匙、重复执行、越权操作，会造成流程成本、数据损坏和业务风险。

1.2 Agentic AI 的宽口径定义

视频指出，常见误解是把 agentic AI system 看成 “a large language model with access to tools” [00:00:38–00:00:48]。这个说法只抓住了工具访问这一层。讲者随后补充，agentic AI 是更大的系统，能够 “*Observe*”，并以 “*cyclical or iterative format*” 工作，从而推动更稳定的结果 [00:00:51–00:01:16]。

可以把它理解成一个闭环：

环节	工程含义
Observe	观察任务、环境、工具返回和当前状态。
Plan	把目标拆成步骤，并估计下一步行动。
Act / Tool Use	调用搜索、数据库、邮件、预订、文件系统等外部能力。
Evaluate	判断结果是否满足目标，是否需要重试或升级。
Retry / Stop	根据停止条件、进度信号和风险规则决定继续、停止或交给人。



图 1: Agentic AI 的基本结构：LLM 连接工具，并通过 plan-act-observe-adapt 的迭代回路逐步接近目标。¹

¹视频画面时间区间：00:00:38–00:01:16。若图像为裁剪图，时间区间仍对应原视频画面。

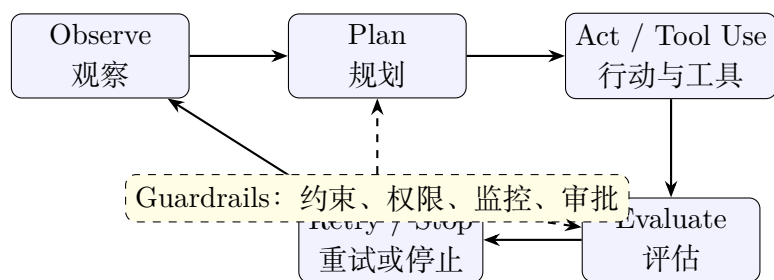


图 2: 一个可审计的 agentic AI 回路。该图是教学重绘，依据视频在 00:00:38–00:01:16 对 observe 与 cyclical/iterative format 的说明组织。

1.3 失败面为什么从 chatbot 扩大到 agentic system

简单 chatbot 的主要输出是文本。它的失败通常表现为答错、漏答、编造来源。Agentic system 会对外部环境施加动作：搜索文档、发邮件、预订机票、修改数据库、调用 API。动作带来副作用，副作用带来新的失败模式。

视频在 [00:01:16–00:01:25] 明确说，因为系统变复杂，今天看到的失败类型比过去简单聊天应用更多。这个判断非常关键。复杂性集中体现在组件之间的反馈回路：反馈回路如果没有刹车，就会进入无限循环；计划如果没有能力校验，就会在执行阶段崩掉；工具如果权限过大，就会把一次错误判断放大成业务事故。

常见误判

把 agentic AI 失败笼统叫作“模型幻觉”，会让团队继续调 prompt、换模型、加 few-shot 示例，却没有补上停止条件、进度指标、工具 schema、审批流和权限分层。结果是表面输出更顺滑，系统风险仍然留在原地。

1.4 本章小结

本章建立了整份笔记的分析框架。开场的 “*the model probably hallucinated*” 是常见直觉；“*probabilistic models*” 和 “*not deterministic*” 解释了这个直觉的来源。视频的主线随后转向 “*flaws in the system design*”：agentic AI 是带工具、状态、计划、执行、评估和重试的复合控制系统。接下来三章分别分析三个系统性失败：Infinite Loop、Hallucinated Planning 和 Unsafe Tool Use。

2 失败模式一：Infinite Loop（无限循环）

第一个失败模式是 “*the infinite loop*” [00:01:41–00:01:46]。讲者给出的定义很直接：智能体重复执行相同或相似任务，却 “*without making any meaningful progress*”，无法实质性推进目标完成 [00:01:53–00:02:02]。这类失败听起来像程序 bug，实际更像控制系统缺少停止规则：马达一直转，传送带没有货，电费照样上涨。

无限循环的本章定义

Infinite Loop 指智能体在 search、evaluate、plan、retry 等动作之间反复切换，行动看似持续发生，目标距离却没有缩短。核心缺口通常是 termination condition、action tracking 和 progress tracking。

2.1 文档搜索例子：search、evaluate、plan、retry

视频用“找一份文档”解释无限循环 [00:02:05–00:03:24]。用户让智能体找文档，智能体先配置搜索，再调用 search tool，得到结果后做 evaluate。如果结果看起来好，就给答案；如果结果看起来差，就重新 plan，改写搜索词，再 retry。

真正的问题出现在一个隐藏条件上：这份文档根本不存在 [00:02:56–00:03:16]。智能体不知道这个事实。每次搜索都得到一些“pretty vague”的结果 [00:03:02–00:03:06]，这些模糊结果又足以诱发它继续重试。于是它陷入“*retrying and planning, evaluating, and searching*”的回路 [00:03:16–00:03:23]。

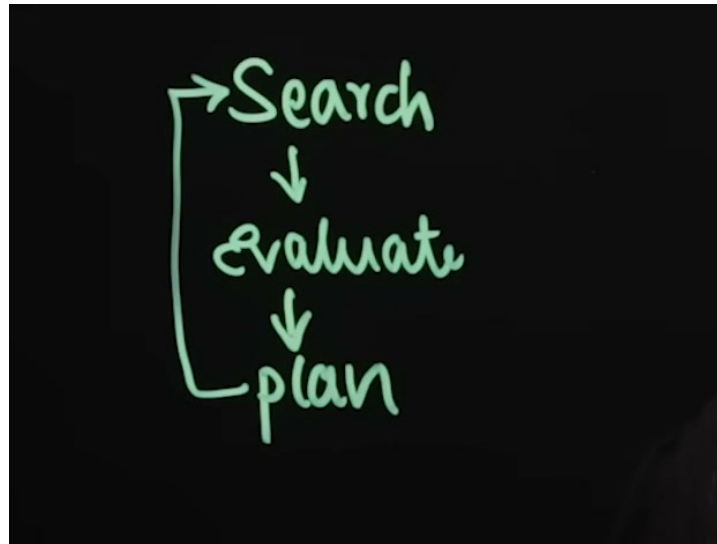


图 3: 无限循环的核心形态：Search、evaluate、plan 反复回到起点，重试行为被误当成任务进展。²

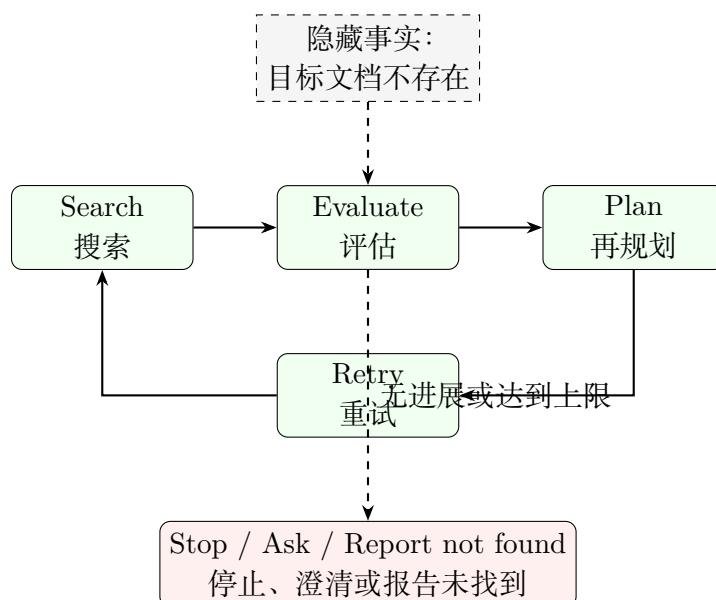


图 4: 缺少终止条件时，系统会把“继续尝试”误读成“仍有希望”；加入退出路径后，失败状态也能被明确处理。该图是教学重绘，依据 00:02:05–00:03:26 的文档搜索例子组织。

²视频画面时间区间：00:02:05–00:03:26。若图像为裁剪图，时间区间仍对应原视频画面。

直觉类比：关着门找钥匙

如果钥匙确实不在房间里，换十种路线在同一个房间里找，成功概率也不会凭空变高。无限循环的危险就在这里：系统把“动作仍在发生”误读成“任务仍有希望”，于是把时间、token、API 调用和用户等待都消耗掉。

2.2 根因一：缺少 termination condition

讲者说，主要原因之一是没有“*proper termination condition*” [00:03:26–00:03:32]。termination condition 可以译为“终止条件”或“停止规则”：什么时候认定当前路径已经没有收益，什么时候应该停止、升级、询问用户、报告找不到。

视频把它解释得更口语化：智能体“*doesn't know when to stop trying*” [00:03:42–00:03:45]。这句话应该成为所有 agentic system 的设计检查项。没有停止规则，retry 机制会从容错手段变成成本放大器。

重试有明确适用边界

重试只对偶发性失败有效，例如临时网络超时、服务端限流、搜索索引短暂不可用。若失败来自目标不存在、权限缺失或工具能力不匹配，更多重试只会更快耗尽预算。

2.3 根因二：缺少 action tracking

第二个根因是缺少 action tracking。讲者说，每次 retry 时，系统无法确认智能体是否真的用不同方式搜索；如果“*not tracking the action*”，团队就无法判断行动是否发生了实质变化 [00:03:46–00:04:03]。

action tracking 记录的是“智能体做了什么”：搜索关键词、过滤条件、调用工具、参数、时间、返回状态。它回答的问题是：这次行动和上次行动是否显著不同？若每次只是把 *quarterly report* 改成 *quarterly document*，搜索空间几乎没有变化，继续重试没有工程价值。

2.4 根因三：缺少 progress tracking

第三个根因是缺少 progress tracking。视频说，系统需要知道每次 retry 后结果是否更好；如果不追踪，就无法判断 retry 是否成功 [00:04:11–00:04:32]。progress tracking 记录的是“目标距离是否变小”：相似度、命中率、置信度、候选数量、人工标注反馈、错误类型是否收敛。

一个轻量教学抽象可以这样写。先用普通话说：把每次搜索结果打一个分，比较本次分数与上次分数。如果连续没有提升，系统应该停止、升级或请求澄清。

$$\Delta_t = \text{score}(r_t) - \text{score}(r_{t-1})$$

- Δ_t ：第 t 次 retry 相比上一次的进展变化。
- t ：迭代编号，例如第 1 次、第 2 次、第 3 次搜索。
- r_t ：第 t 次行动后的结果，例如搜索返回列表。
- r_{t-1} ：上一次行动后的结果。
- $\text{score}(\cdot)$ ：结果质量评分函数，可以来自规则、模型、检索相似度或人工反馈。

当连续多次 $\Delta_t \leq 0$ ，并且 retry 次数达到 $N_{\max}$ 时，系统应该停止当前路径。这里的公式是笔记作者的工程抽象，视频本身给的是设计建议 [00:05:29–00:05:48]。

2.5 缓解：上限、差异比较与进度指标

视频给出三类缓解措施。第一类是直接设置停止条件：max retries、max number of steps、max runtime usage [00:04:37–00:04:56]。这些上限像保险丝，目标是防止系统无限消耗 compute 和 API 成本。讲者明确提到 “wasting resources like compute and the API costs are going up” [00:04:56–00:05:05]。

第二类是 action tracking：比较当前行动和历史行动，看它们是否 “significantly different” [00:05:06–00:05:16]。若所有 retry 都使用相同或高度相似的搜索条件，继续消耗 compute 没有意义 [00:05:16–00:05:24]。

第三类是 progress tracking：确保每次 retry 都带来更好的结果 [00:05:29–00:05:42]。若没有改善，系统就应该停止当前策略。这一点会在最终综合章节再次出现，因为 monitoring 和 tracking 是三个失败模式共同需要的底层能力。

下表使用三个局部符号： $N_{\max}$ 表示最大重试次数， $S_{\max}$ 表示最大执行步数， $T_{\max}$ 表示最大运行时间。它们对应视频里的 max retries、max number of steps 和 max runtime usage [00:04:37–00:04:56]。

根因	失败表现	守护栏
缺少终止条件	不知道何时停止尝试	$N_{\max}$ 、 $S_{\max}$ 、 $T_{\max}$ 和升级路径
缺少动作追踪	看不出 retry 是否换了策略	记录查询、工具、参数，并比较相邻行动差异
缺少进度追踪	看不出结果是否变好	结果评分、候选质量、错误收敛度、无进展阈值

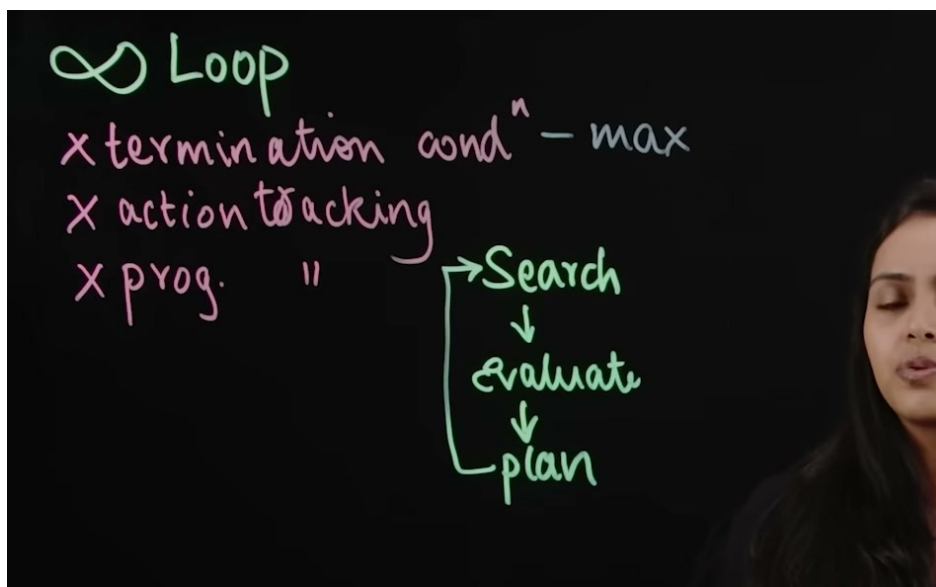


图 5: 打断无限循环的三类控制：终止条件、动作差异追踪、进展追踪。³

³视频画面时间区间：00:03:26–00:05:28。若图像为裁剪图，时间区间仍对应原视频画面。

2.6 可复用失败模式卡片

Symptom	智能体一直搜索、评估、重新计划、再次搜索，用户看见系统忙碌却没有得到答案。
Hidden cause	缺少停止条件、动作追踪、进度追踪，且系统没有把“文档不存在”作为可能状态处理。
Trigger example	文档搜索任务中，目标文档不在系统里，模糊搜索结果诱发反复 retry [00:02:56–00:03:23]。
Guardrail	设置最大重试、最大步数、最大运行时间；比较行动差异；用进展指标决定是否升级。
Residual risk	即使有上限，系统仍可能在上限内浪费成本；若评分函数设计差，progress tracking 会给出虚假改善。

2.7 本章小结

Infinite Loop 是一种控制流失败。视频里的文档搜索例子说明，若目标不存在，更多搜索不会自动产生目标。系统需要 “*proper termination condition*”，需要追踪行动是否真的改变，也需要追踪结果是否改善。工程上可以用 $N_{\max}$ 、 $S_{\max}$ 、 $T_{\max}$ 和 Δ_t 这类轻量指标把“继续尝试”变成可审计决策。

按视频的严重性校准，这个模式通常低于破坏性工具调用；主要损失是 compute、API 成本和用户等待时间持续上升 [00:05:51–00:06:04]。这种成本型失败仍值得设计阶段认真处理，因为它会在规模化部署后被调用量放大。

3 失败模式二：Hallucinated Planning（幻觉式规划）

第二个失败模式是 “*hallucinated planning*” [00:06:05–00:06:10]。它和第一章提到的普通模型幻觉有联系，焦点却落在计划层：智能体给出一串听起来很合理的步骤，执行时发现工具、权限、输入或审批条件不存在。

视频用一句高价值英文抓住核心差别：“*plausible versus possible*” [00:06:17–00:06:25]。plausible 是“听起来合理、符合常识脚本”；possible 是“在当前系统能力和约束下真实可执行”。随后讲者补充，这种计划 “*looks great on paper, but fails at execution*” [00:06:25–00:06:30]。这里的 *plane* 是字幕小错误，应按上下文理解为 plan。

本章核心定义

Hallucinated Planning（幻觉式规划）指计划看起来连贯，实际假设了系统没有的能力。它的根因是计划与工具能力、输入信息、权限边界和约束条件之间发生错配。

3.1 米兰机票例子：漂亮计划与真实能力

讲者给出的例子是：用户要求智能体预订去 Milan 的机票，价格低于 \$500 [00:06:33–00:06:37]。智能体很快生成一个 “beautiful plan”：使用 travel booking API 搜索机票，设置 under \$500 的过滤条件，完成预订，再发送确认邮件 [00:06:38–00:06:54]。

这个计划符合人类对旅行预订的常识脚本，所以 plausible。执行失败的原因也很具体：系统可能没有配置 travel API 访问权限，用户可能没有提供 email address，智能体甚至可能没有 email tool [00:06:54–00:07:12]。讲者把这种现象总结为：智能体 “*assuming capabilities instead of relying on what it actually has access to*” [00:07:13–00:07:22]。

检查项	Plausible plan	Executable plan
工具能力	计划里写了会调用 travel API	系统实际配置了可用 API, 并有明确 schema
必要输入 权限边界	假设知道邮箱、日期、偏好 默认可以预订、发送邮件	输入缺失时先澄清, 再进入执行 预订和发送需权限、审批或人确认
验证状态	计划生成后直接执行	计划先经过 verifier 或 human review

直觉类比：纸上行程与手里证件

一个人可以在纸上写出“明天飞米兰、住酒店、会议后发邮件”的完美行程。若他没有护照、没有订票账号、没有公司付款权限，这份行程只能说明他懂旅行流程，无法说明他能完成旅行任务。Hallucinated planning 的危险也在这里：流畅计划容易给人一种“系统已经会做”的错觉。

3.2 根因一：tool capabilities 没有清晰定义

讲者说，第一类原因是 “*tool capabilities are not well-defined*” [00:07:25–00:07:31]。智能体不知道工具能做什么、不能做什么 [00:07:36–00:07:39]。在工程上，这经常表现为工具描述过短、schema 模糊、错误返回不清楚、权限边界没有暴露给规划器。

tool schema 可以译为“工具接口规范”。视频要求 “*Clearly define the tool schema*”，让智能体理解工具的 capabilities and limitations [00:08:28–00:08:34]。一个实用 schema 至少要说明输入字段、字段类型、必填项、返回结构、错误类型、速率限制、权限要求、禁止动作和需要人工确认的条件。

模糊 schema 会制造假能力

如果工具描述只写 “book travel”，智能体可能默认它能搜索、筛选、支付、出票、发邮件。若实际工具只能查询公开航班信息，后面所有预订和邮件步骤都属于能力假设。

3.3 根因二：planning 与 execution 缺少 validation gate

第二类原因是把 planning 和 execution 连在一起执行，期间没有 validation。讲者说，团队让智能体 plan and execute without separating those two things out [00:07:40–00:07:46]，于是计划在执行前没有被验证 [00:07:47–00:07:57]。结果是智能体执行了无法完成的事情，系统才在工具调用阶段报错 [00:07:57–00:08:01]。

可执行计划需要满足一个最小条件。先用普通话说：计划里的每一步都必须映射到真实工具能力，并且满足约束。这个教学抽象可以写成：

$$\forall a_i \in P, \quad a_i \in C_{\text{tools}} \wedge a_i \models C_{\text{constraints}}$$

- P ：智能体生成的计划。
- a_i ：计划中的第 i 个行动，例如搜索航班、设置价格过滤、预订、发送邮件。
- C_{tools} ：当前系统真实可用的工具能力集合。
- $C_{\text{constraints}}$ ：当前任务的约束集合，例如预算、权限、用户输入、审批策略、合规规则。

- $\forall$: 表示“对所有”。
- $\in$: 表示“属于某个集合”。
- $\wedge$: 逻辑“并且”。
- $\models$: 表示“满足约束”。

这个公式来自工程抽象，视频本身用 travel API、email address 和 email tool 说明同一件事 [00:06:54–00:07:12]。

3.4 根因三：先假设能力，后考虑 constraints

第三类原因是智能体先假设能力，再处理约束。讲者说，它会 “*assuming capabilities instead of checking for constraints*” [00:08:03–00:08:08]。这句话把 planning 的顺序问题说得很清楚：可靠系统应该先识别边界，再生成行动；脆弱系统会先生成熟悉脚本，再希望工具世界配合它。

这里的 constraints 包括工具可用性、凭证、用户授权、预算、时间、地域、合规、数据访问范围和高风险动作审批。对真实项目来说，隐藏约束经常比显式工具更重要。一个团队可能给了航班查询 API，却没有支付权限；给了邮件草稿工具，却没有直接发送权限；给了数据库读权限，却没有写权限。

3.5 缓解：清晰 schema、verifier agent、人类复核

第一种缓解是清晰描述工具：“*Clearly describe what the tools can cannot do*” [00:08:26–00:08:28]，并定义 “*tool schema*” [00:08:28–00:08:34]。这里的中文写法应该规范成“工具能做什么、不能做什么”。工具描述需要像合同条款，少用“智能地处理”“自动完成”这种宽泛词。

第二种缓解是引入验证架构。讲者提到 multi-agent 架构，在 planning 和 executing 之间放一个 “*verifier agent*” [00:08:35–00:08:53]。verifier agent 的职责是检查计划能否执行：工具是否存在、输入是否齐全、权限是否满足、风险是否需要审批。对更严肃或高风险的计划，可以加入 “*human in the loop*” [00:08:53–00:09:00]。

第三种缓解是明确约束，并要求智能体 “*ask for clarification before making assumptions*” [00:09:15–00:09:20]。视频给出的示例是让智能体先问用户：是否希望它使用 travel API，而直接虚构一个 travel API 会把错误推迟到执行阶段 [00:09:20–00:09:28]。

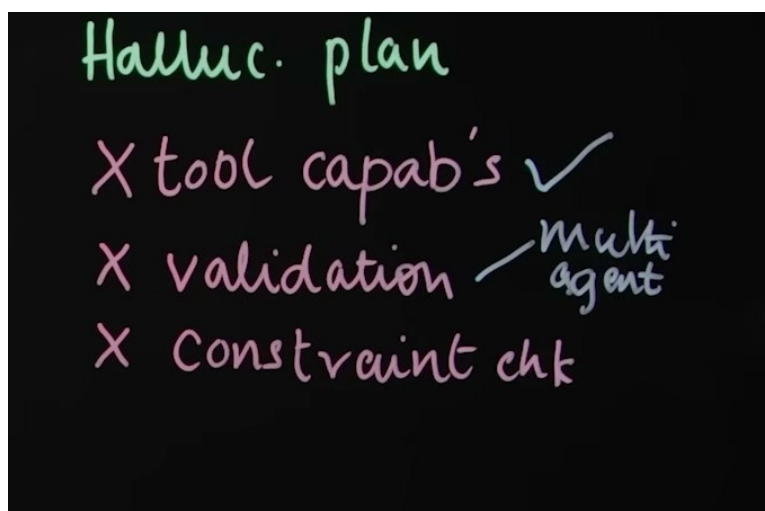
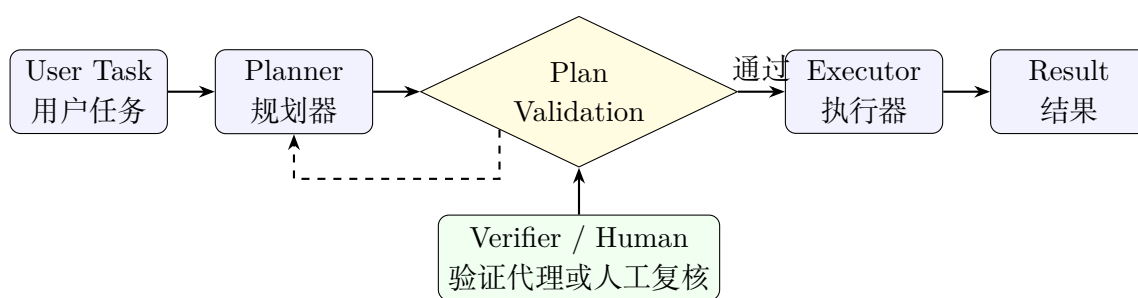


图 6: 幻觉式规划的防线：明确工具能力、验证计划、检查约束，并可用 verifier agent 承担中间审查。⁴



不通过：澄清或重写计划

图 7: 在规划和执行之间加入验证门，可以把“听起来合理”的计划转化为“系统实际可执行”的计划。该图是教学重绘，依据 00:08:20–00:09:37 的 tool schema、verifier agent、human in the loop 与 constraints 建议组织。

3.6 可复用失败模式卡片

Symptom	计划看起来专业、完整、步骤清楚，执行时出现工具不存在、权限不足、输入缺失或审批缺失。
Hidden cause	计划没有经过能力和约束验证；工具 schema 没有把真实边界暴露给规划器。
Trigger example	米兰机票任务中，智能体假设 travel booking API、email address 和 email tool 都可用 [00:06:33–00:07:22]。
Guardrail	定义工具 schema；把 planning 与 execution 分层；使用 verifier agent 或 human in the loop。
Residual risk	verifier 自身也可能读错 schema；人类审批若只看摘要，仍可能漏掉关键能力假设。

3.7 本章小结

Hallucinated Planning 的核心是 “*plausible versus possible*”。一个计划可以看起来符合常识，却因为缺少工具、凭证、输入或审批而无法执行。视频的米兰机票例子说明，漂亮计划需要经过能

⁴视频画面时间区间：00:07:23–00:09:37。若图像为裁剪图，时间区间仍对应原视频画面。

力集合 C_{tools} 和约束集合 $C_{\text{constraints}}$ 的验证。工程上应使用清晰 tool schema、verifier agent、人类复核和澄清提问，把“会说计划”推进到“能执行计划”。下一章的不安全工具使用会把这里的权限与审批问题继续推进：当计划真的进入执行阶段，权限过大和审批缺口会把一次错误计划放大成业务风险。

4 失败模式三：Unsafe Tool Use（不安全工具使用）

第三个失败模式是“unsafe tool use”[00:09:37–00:09:44]。它讨论的错误更加尖锐：智能体执行了一个在技术上可调用的动作，动作本身可能通过 API 校验，却会造成风险、破坏或非预期后果。讲者的定义是：an action that is “technically valid, but could be risky, destructive, or unintended”[00:09:49–00:09:57]。

这一章的关键差别在于“有效”和“安全”。有效关注工具调用格式、权限令牌、参数类型是否通过；安全关注这件事在业务语境里是否应该发生。删除语句可以合法执行，仍可能删除错数据。邮件发送 API 可以返回成功，仍可能把未审核内容发给客户。

本章核心定义

Unsafe Tool Use 指工具调用在接口层面成立，业务层面风险过高。它通常来自 overprivileged tools、缺失 approval workflow，以及 read/write/delete 权限分层不清。

4.1 数据库例子：删除 outdated records 时误删 active records

视频的第一个例子是数据库删除：智能体本来应该“delete outdated records”，也就是删除过期记录；结果它删除了“active records”，也就是仍然重要的活跃记录[00:10:11–00:10:23]。这里没有必要假设工具调用语法错了。更有可能的风险是筛选条件、状态定义、权限范围和审批流程设计得太粗。

这个例子在企业系统里很真实。很多业务表的“过期”“归档”“活跃”往往无法只靠单字段判断。订单可能已完成但仍在财务稽核期；用户账号可能长期未登录但仍是付费客户；日志可能超过保留期限却涉及合规调查。让智能体直接执行删除，会把语义不确定性转化成不可逆副作用。

4.2 邮件例子：未 review 的 autonomous emails

第二个例子是邮件。讲者提到智能体发送“autonomous emails”或 automated emails，内容“has not been reviewed”[00:10:24–00:10:32]。这里的危险范围远超礼貌问题，现实风险包括泄露客户信息、承诺错误价格、触发合规问题、损害品牌声誉，或者把内部判断发给外部收件人。

视频随后指出，这类问题也会在没有“approval workflow”的情况下发生[00:10:32–00:10:37]。approval workflow 可以译为审批流程或批准工作流：高风险动作在执行前经过人、策略引擎或独立验证器检查。

“能调用”与“该调用”的差别

工具返回成功只说明接口接受了动作。它没有自动证明业务对象选对了、上下文完整了、用户同意了、后果可逆了。真实系统要把动作的业务风险显式建模。

4.3 根因一：overprivileged tools

讲者说，unsafe tool use 主要因为 “tools are overprivileged” [00:09:58–00:10:02]。overprivileged tools 指工具拥有超过任务需要的权限。例如一个“清理过期记录”的工具同时有全表读、全表写和全表删除能力；一个“生成邮件草稿”的工具同时有直接发送能力。

权限过大时，模型的一次判断偏差会直接变成破坏性动作。最坏的情况在于工具过度强大：它把读、写、删、发送、支付、发布都包进同一个入口，让规划器在不充分语境下越过安全边界。

4.4 根因二：approval workflow 缺失

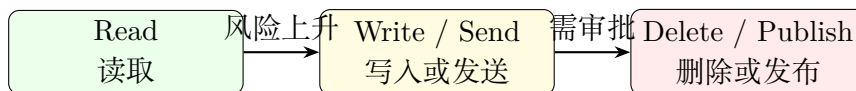
第二个根因是缺少 approval workflow。视频在示例后立刻指出，没有合适审批流会导致未审核内容或高风险操作直接执行 [00:10:32–00:10:37]。审批流的意义在于识别风险动作：风险高、后果难撤销、对外可见或涉及敏感数据的动作需要在执行前被拦住。

对 agentic AI，审批对象至少包括三类：计划审批、参数审批、执行审批。计划审批检查“要不要做”；参数审批检查“对谁做、用什么条件做”；执行审批检查“是否已经满足可执行前置条件”。第三章的 verifier agent 也可以承担一部分审批前验证，因此这两章存在天然交叉。

4.5 根因三：read、write、delete 权限层级混在一起

视频还提到，问题会在 read 和 write 访问边界不清时发生 [00:10:43–00:10:50]，随后字幕把 access 误写成 “read axis or write axis or a delete axis” [00:11:40–00:11:45]。这里应规范理解为 read/write/delete access tiers，即读、写、删除三类权限层级。

三类权限的风险差距很大。read 可能泄露信息；write 可能污染状态；delete 可能造成不可逆损失。更完整的安全设计还应考虑 send、publish、pay、deploy 这类外部副作用动作。它们和 delete 一样，需要更高等级的审批和审计。



权限层级越高，越需要审批、审计、回滚和人工确认。

图 8: 工具权限应按动作风险分层：读、写、删除或发布代表不同级别的业务后果。该图是教学重绘，依据 00:10:43–00:11:50 的 read/write/delete 权限分层建议组织。

least agency 与 least privilege

视频使用 “principle of least agency” [00:11:03–00:11:07]。它可以译为“最小代理性原则”：给智能体的自主行动空间应缩到任务真实需要的最小范围。它与安全领域的 principle of least privilege（最小权限原则）相邻。least privilege 关注权限集合，least agency 还关注自主决策范围、工具副作用和是否需要人在环。

4.6 风险门：把安全性做成可执行规则

视频给的是设计建议，工程落地时可以用一个轻量风险门表达。先用普通话说：若行动风险低于阈值、权限匹配、审批齐全，则允许执行；若风险超过阈值或缺少审批，则拒绝、降级或进入人工流程。

$$\text{allow}(a) = \begin{cases} 1, & \text{risk}(a) \leq \tau \text{ and permissions match} \\ 0, & \text{risk}(a) > \tau \text{ or approval is missing} \end{cases}$$

- $\text{allow}(a)$: 动作 a 是否允许执行, 1 表示允许, 0 表示拦截。
- a : 待执行动作, 例如删除记录、发送邮件、写数据库。
- $\text{risk}(a)$: 动作 a 的风险评分, 可以由动作类型、数据敏感性、可逆性、外部影响决定。
- τ : 风险阈值, 超过该阈值时需要更高等级保护。
- $\leq$: 小于或等于。
- $>$: 大于。
- permissions match: 当前工具权限与任务所需权限一致。
- approval is missing: 缺少必要审批。

4.7 缓解：最小代理性、审批流、权限分层

视频给出三类缓解。第一, 给工具 “*only those privileges that are needed*” [00:10:58–00:11:03], 采用 “*principle of least agency*” [00:11:03–00:11:07]。第二, 为高风险任务创建 “*proper approval workflow*”, 必要时让 “*human in the loop*” 在执行前 review [00:11:20–00:11:33]。第三, 按访问类型把工具分层: read、write、delete [00:11:34–00:11:50]。



图 9: 不安全工具调用的治理路径: 限制过度权限、建立审批流程、区分读写权限, 并落实最小代理性原则。⁵

⁵视频画面时间区间: 00:09:49–00:12:04。若图像为裁剪图, 时间区间仍对应原视频画面。

动作层级	典型风险	推荐守护栏
read	敏感数据泄露、越权查看	作用域限制、脱敏、审计日志
write	状态污染、错误更新、重复写入	dry-run、参数验证、幂等键、回滚方案
delete	不可逆损失、合规风险、业务中断	人工审批、软删除、备份、双人复核
send/publish	对外承诺、品牌损害、信息泄露	草稿模式、内容审核、收件人确认、发送延迟窗口

4.8 可复用失败模式卡片

Symptom	工具调用成功，结果却造成数据误删、未审核外发、错误业务动作或公司声誉损害。
Hidden cause	工具权限过大、审批流程缺失、访问层级混乱，系统没有把业务风险纳入执行前判断。
Trigger example	删除 outdated records 时误删 active records；发送未 review 的 autonomous emails [00:10:11–00:10:32]。
Guardrail	最小代理性原则、审批 workflow、read/write/delete 分层、审计和回滚策略。
Residual risk	人工审批可能流于形式；软删除若没有恢复演练，事故发生时仍可能恢复失败。

4.9 本章小结

Unsafe Tool Use 讨论的是“接口有效、业务危险”的失败。视频用数据库删除和自动邮件说明，技术上可执行的动作仍可能 *“risky, destructive, or unintended”*。可靠设计需要限制 overprivileged tools，引入 approval workflow，并把 read、write、delete 等访问层级分开。最小代理性原则提醒团队：智能体的自主空间、工具权限和破坏性动作都应缩到任务真实需要的范围内。讲者在收束处强调，这类失败会损害公司的声誉，因此系统设计必须主动把它纳入权限、审批和监控框架 [00:11:51–00:12:04]。

5 总结与工程检查清单

视频最后把三类失败压缩成一个清晰判断：*“Agentic AI failures are not random occurrences”*，它们是可预测的 [00:12:04–00:12:09]。讲者随后给出原因：*“too much autonomy or too little constraint”* [00:12:11–00:12:16]，以及 *“no proper monitoring or tracking in place”* [00:12:18–00:12:22]。最后一句实质性收束是：*“engineering discipline is key to building reliable agents”* [00:12:24–00:12:29]。

整段视频的压缩结论

Agentic AI 可靠性来自三个配套能力：约束自主性、验证可执行性、追踪运行过程。自主性越高，停止条件、能力校验、权限分层、审批流和监控指标越需要同步增强。

5.1 三类失败的共同结构

三个失败模式表面差别很大，结构上高度一致。Infinite Loop 问的是：智能体知道什么时候进展停滞吗？Hallucinated Planning 问的是：计划是否匹配真实工具能力与约束？Unsafe Tool Use 问的是：每个行动是否拥有合适权限、审批和风险边界？

Failure Mode	核心问题	主要守护栏
Infinite Loop	重试是否带来真实进展	termination condition、action tracking、progress tracking
Hallucinated Planning	计划是否真实可执行	tool schema、constraint validation、verifier agent、human in the loop
Unsafe Tool Use	动作是否安全且被授权	least agency、approval workflow、read/write/delete access tiers

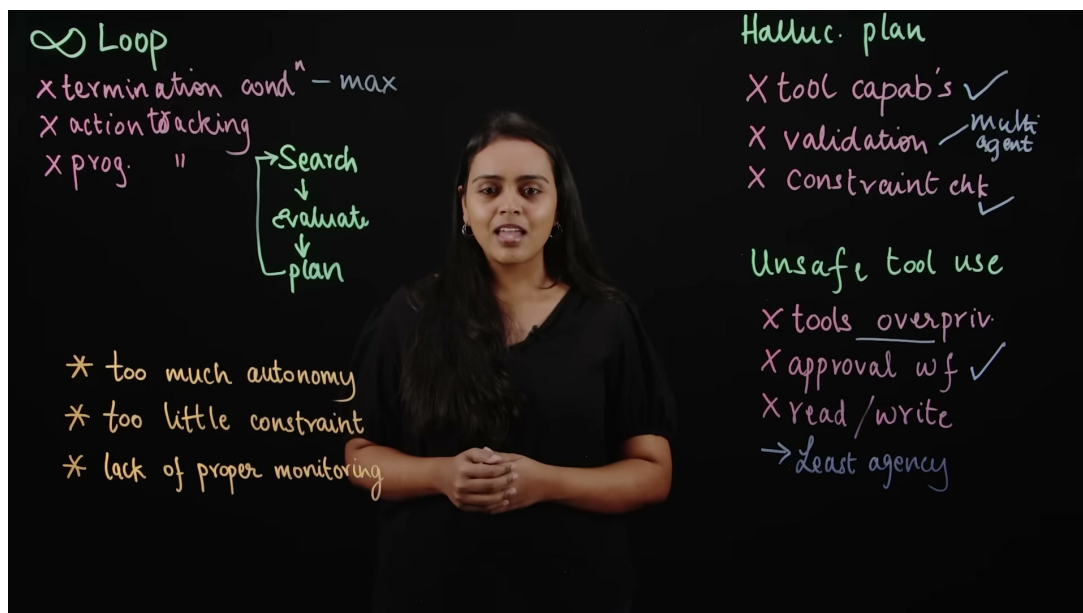


图 10: 代理系统失败的三类底层驱动：过多自治、过少约束、缺少监控。⁶

5.2 too much autonomy or too little constraint

Autonomy 可以译为自主性：系统可以自己观察、计划、调用工具、评估和重试。自主性带来效率，也带来错误放大。若一个 agent 只能回答问题，错误多停留在文本层；若它还能删数据、发邮件、预订、付款、发布内容，错误会进入业务世界。

Constraint 可以译为约束：工具范围、权限范围、预算上限、运行时间、审批条件、风险阈值、停止条件、用户确认。约束像护栏，作用在于把系统能力限定在可接受风险区间内，同时避免能力无限扩张。

一个轻量可靠性抽象可以这样写：

$$R_{\text{agent}} = f(B_{\text{stop}}, V_{\text{plan}}, P_{\text{perm}}, M_{\text{track}})$$

- R_{agent} ：agentic system 的可靠性水平。
- $f(\cdot)$ ：表示可靠性由多个工程因素共同决定的函数。
- B_{stop} ：停止边界，包括最大重试、最大步数、最大运行时间和无进展阈值。
- V_{plan} ：计划验证能力，包括 schema 校验、约束检查、verifier agent 或人工复核。

⁶视频画面时间区间：00:12:04–00:12:32。若图像为裁剪图，时间区间仍对应原视频画面。

- P_{perm} : 权限设计, 包括最小代理性、访问层级和高风险动作审批。
- M_{track} : 监控与追踪, 包括动作日志、进度指标、成本、错误类型和审计记录。

这个公式是笔记作者的总结抽象, 用于把视频的工程建议组织成可检查框架。

未知盲点: 约束也需要版本管理

许多团队会版本管理 prompt 和模型, 却忽略工具 schema、权限策略、审批阈值和停止条件的版本。真实事故追踪时, 团队需要回答“当时哪版工具描述生效”“当时 delete 权限给了谁”“当时 max retries 是多少”。没有这些记录, 复盘会变成猜测。

5.3 monitoring 与 tracking 的底层作用

视频最后点名 “*monitoring or tracking*” [00:12:18–00:12:22]。这两个词经常被混用, 工程上可以稍作区分。tracking 更偏记录: 每一步做了什么、输入输出是什么、结果是否改善。monitoring 更偏观察和告警: 成本是否异常、失败率是否上升、高风险动作是否排队、审批是否被绕过。

对 Infinite Loop, tracking 让系统知道 retry 是否换了策略, monitoring 让团队发现 API 成本突然上升。对 Hallucinated Planning, tracking 记录计划验证失败点, monitoring 观察某类工具能力假设是否频繁出现。对 Unsafe Tool Use, tracking 记录谁触发了什么动作, monitoring 对 delete、send、publish 等高风险动作发出告警。

没有日志就没有可靠性工程

Agentic AI 的很多失败发生在“过程”里, 而用户只看到最终失败结果。若没有 action log、tool call log、plan validation log 和 approval log, 团队无法区分模型误判、schema 不清、权限过大、进度停滞和审批缺失。

5.4 从 prompt 调参走向系统可靠性

本视频隐含了一个成熟工程转向: prompt 调参只能处理一部分语言行为, 系统可靠性需要控制回路、验证门和权限门。更具体地说:

- 当问题是 Infinite Loop, 优先检查 stop rule、retry policy、progress metric。
- 当问题是 Hallucinated Planning, 优先检查 tool schema、capability registry、plan validation。
- 当问题是 Unsafe Tool Use, 优先检查 permission tier、approval workflow、audit and rollback。

模型质量依然重要; 系统设计决定模型输出如何转化成外部动作。高质量模型接入脆弱执行系统, 仍会在边界条件里失败。

5.5 面向真实项目的检查清单

下面的清单可作为生产 agent 评审的最小版本。

检查项	需要回答的问题
目标与退出	目标 G 是否明确？找不到、无权限、无进展时系统如何停止或升级？
重试控制	是否设置 $N_{\max}$ 、 $S_{\max}$ 、 $T_{\max}$ ？重试是否要求行动差异和进展提升？
工具 schema	每个工具的输入、输出、错误、权限、禁止动作和限制是否清楚？
计划验证	执行前是否检查 $a_i \in C_{\text{tools}}$ 且 $a_i \models C_{\text{constraints}}$ ？
权限分层	read、write、delete、send、publish 是否分成不同工具或不同权限等级？
审批流程	高风险动作是否进入 approval workflow？审批内容是否包含参数和后果？
监控追踪	是否记录 action、result、cost、risk、approval、error type？是否有异常告警？
恢复能力	是否支持 dry-run、软删除、回滚、重放、人工接管和事故复盘？

检查维度	关键问题	通过标准
Loop control	系统何时停止、升级或澄清？	有最大步数、最大运行时间、无进展阈值和审计记录
Plan validation	每个计划步骤是否有真实工具和输入？	工具 schema、约束检查、verifier 或人工复核到位
Permission safety	工具动作的业务风险是否被限制？	读写删除分层、审批流程、最小代理性原则、回滚路径完整
Monitoring	团队能否复盘 agent 做了什么？	动作、参数、结果、进展指标和人工决策均可追踪

5.6 本章小结

本章把视频的三类失败统一到一个可靠性框架里。Infinite Loop 暴露停止和进度追踪缺口；Hallucinated Planning 暴露能力和约束验证缺口；Unsafe Tool Use 暴露权限和审批缺口。讲者最后强调，agentic AI failures are predictable, 根源常在 “*too much autonomy or too little constraint*”，并且缺少 “*monitoring or tracking*”。可靠 agent 依赖工程纪律：用边界管理自主性，用验证连接计划和执行，用权限与审批控制副作用。